

Lab 2- Tic Tac Toe

In this lab your will build a $n \times n$ Tic Tac Toe game. As you do the exercises, make sure your solutions work for any size Tic Tac Toe game.

Assignment Submission Guidelines

Steps to Submit Your Lab Assignments Using Google Colab

1. Submission Deadline:

- All assignments must be submitted **no later than 23:59 PM tonight**.

2. Complete Your Work:

- Finish your work in Google Colab and ensure that your notebook is saved.

3. Submit Your Assignment on Canvas:

- Log in to **Canvas** and navigate to the assignment submission page.

4. File Naming Convention:

- Ensure that you naming is as follows: `Lastname_Firstname_AssignmentName`
- Example: `Alex_John_Lab2.ipynb` and `Alex_John_Lab2.pdf`

5. Technical Issues:

- If you encounter any technical issues with Canvas or your submission, contact the TAs immediately **before the deadline** to avoid penalties.

Exercise 1: Write a function that creates an n by n matrix (of list of lists) which will represent the state of a Tie Tac Toe game. Let 0, 1, and 2 represent empty, "X", and "O", respectively.

```
In [121... def create_board(board_size):  
    if type(board_size) != int or board_size < 1:  
        raise ValueError('Please check your input.')  
    return [[0] * board_size for _ in range(board_size)]
```

```
In [122... game_board = create_board(3)  
game_board
```

```
Out[122... [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

Exercise 2: Write a function that takes 2 integers `n` and `m` as input and draws a `n` by `m` game board. For example the following is a 3x3 board:

```
--- --- ---  
|   |   |   |  
--- --- ---  
|   |   |   |  
--- --- ---  
|   |   |   |  
--- --- ---
```

```
In [123... def draw_game_board(n, m):
    top_divider = '+---' * m + '+'
    side_divider = '|   ' * m + '|'
    for row in range(n):
        print(top_divider)
        print(side_divider)
    print(top_divider)
```

```
In [124... print('I use the + to make sure the alignment is good.')
draw_game_board(3,3)
```

I use the + to make sure the alignment is good.

```
+---+---+---+
|   |   |   |
+---+---+---+
|   |   |   |
+---+---+---+
|   |   |   |
+---+---+---+
```

Exercise 3: Modify exercise 2, so that it takes a matrix of the form from exercise 1 and draws a tic-tac-tie board with "X"s and "O"s.

```
In [125... def value_to_symbol(value):
    if value == 0:
        return ' '
    elif value == 1:
        return 'X'
    elif value == 2:
        return 'O'

def draw_board(input_board):
    n = len(input_board)
    m = len(input_board[0])
    horizontal_line = '+' + '---+' * m
    for row in range(n):
        print(horizontal_line)
        row_text = '|'
        for col in range(m):
            row_text += ' ' + value_to_symbol(input_board[row][col]) + '|'
        print(row_text)
    print(horizontal_line)
```

```
In [126... print('The game board.')

board = create_board(3)
board[0][0] = 1
board[0][2] = 2
board[1][0] = 1
draw_board(board)
```

The game board.

```
+---+---+---+
| X |   | O |
+---+---+---+
| X |   |   |
+---+---+---+
|   |   |   |
+---+---+---+
```

Exercise 4: Write a function that takes a `n` by `n` matrix representing a tic-tac-toe game, and returns -1, 0, 1, or 2 indicating the game is incomplete, the game is a draw, player 1 has won, or player 2 has one, respectively. Here are some example inputs you can use to test your code:

In [127...

```
def check_game_state(test_game_board):
    # The dimension of our game board.
    list_dim = len(test_game_board[0])

    # Initializing the game state variables.
    winner = None # The win condition in each test will return the winner with this variable.

    # Checking for column-wise win condition.
    element_list = [item for row in test_game_board for item in row] # Nested List comprehension
    # We will iterate over the columns and check if the set is of length one and not zero.
    for m in range(list_dim):
        if len(set(element_list[m::list_dim])) == 1 and element_list[m] != 0:
            return element_list[m]

    # Checking for diagonal win condition.
    diagonal_elements = []
    for element in range(list_dim):
        diagonal_elements.append(test_game_board[element][element])
    if len(set(diagonal_elements)) == 1 and diagonal_elements[0] != 0:
        return diagonal_elements[0]

    # Checking for anti-diagonal win condition.
    anti_diagonal_elements = []
    for element in range(list_dim):
        anti_diagonal_elements.append(test_game_board[element][list_dim - 1 - element])
    if len(set(anti_diagonal_elements)) == 1 and anti_diagonal_elements[0] != 0:
        return anti_diagonal_elements[0]

    # Check row-wise win condition.
    for row in test_game_board:
        if len(set(row)) == 1 and row[0] != 0:
            return row[0]

    # A game is a draw if the game board has no empty spaces and no winner.
    if 0 not in element_list: # If 0 is not in element_list, game board is full.
        return 0

    # Game incomplete condition.
    if winner is None:
        return -1
```

In [128...

```
winner_is_2 = [[2, 2, 0],
               [2, 1, 0],
               [2, 1, 1]]

winner_is_1 = [[1, 2, 0],
               [2, 1, 0],
               [2, 1, 1]]

winner_is_also_1 = [[0, 1, 0],
                    [2, 1, 0],
                    [2, 1, 1]]

no_winner = [[1, 2, 0],
              [2, 1, 0],
              [2, 1, 2]]
```

```

also_no_winner = [[1, 2, 0],
                  [2, 1, 0],
                  [2, 1, 0]]

# Additional game states for testing.
row_winner_1 = [[0, 2, 2],
                [0, 1, 0],
                [1, 1, 1]]

row_winner_2 = [[2, 2, 2, 0],
                [0, 1, 0, 2],
                [2, 2, 2, 2],
                [1, 2, 0, 2]]

draw = [[2, 1, 2],
        [2, 1, 2],
        [1, 2, 1]]

```

In [129...

```

print(f'Winner is 2 test: {check_game_state(winner_is_2)}')
print(f'Winner is 1 test: {check_game_state(winner_is_1)}')
print(f'Winner is 1 test: {check_game_state(winner_is_also_1)}')
print(f'Incomplete test 1: {check_game_state(no_winner)}')
print(f'Incomplete test 2: {check_game_state(also_no_winner)}')
print(f'Winner is row 1 test: {check_game_state(row_winner_1)}')
print(f'Winner is row 2 test: {check_game_state(row_winner_2)}')
print(f'Game is a draw: {check_game_state(draw)}')

```

```

Winner is 2 test: 2
Winner is 1 test: 1
Winner is 1 test: 1
Incomplete test 1: -1
Incomplete test 2: -1
Winner is row 1 test: 1
Winner is row 2 test: 2
Game is a draw: 0

```

Exercise 5: Write a function that takes a game board, player number, and `(x,y)` coordinates and places "X" or "O" in the correct location of the game board. Make sure that you only allow filling previously empty locations. Return `True` or `False` to indicate successful placement of "X" or "O".

In [130...

```

# Initializing the starting game board
ex_5_board = create_board(3)

def modify_game_board(game_board, player_number, coords):
    row, col = coords
    n = len(game_board)
    m = len(game_board[0])
    if not (0 <= row < n and 0 <= col < m):
        return False
    if game_board[row][col] == 0:
        game_board[row][col] = player_number
        return True
    else:
        return False

```

In [131...

```

ex_5_board = create_board(3)

modify_game_board(ex_5_board, 1, (1, 0))
draw_board(ex_5_board)

```

```
print('The game board after one move.')

modify_game_board(ex_5_board, 2, (0, 2))
draw_board(ex_5_board)
print('The game board after two moves.')

print(modify_game_board(ex_5_board, 2, (0, 2))) # False cell already taken
```

```
+---+---+---+
|   |   |   |
+---+---+---+
| X |   |   |
+---+---+---+
|   |   |   |
+---+---+---+
The game board after one move.
+---+---+---+
|   |   | 0 |
+---+---+---+
| X |   |   |
+---+---+---+
|   |   |   |
+---+---+---+
The game board after two moves.
False
```

Exercise 6: Modify Exercise 3 to show column and row labels so that players can specify location using "A2" or "C1".

```
In [158... def draw_board(input_board):
    n = len(input_board)
    m = len(input_board[0])
    horizontal_line = ' + ' + '---+' * m
    header = '   ' + ' '.join(f'{c + 1:^3}' for c in range(m))
    print(header)
    for row in range(n):
        print(horizontal_line)
        label = chr(ord('A') + row)
        row_text = f'{label} | '
        for col in range(m):
            row_text += ' ' + value_to_symbol(input_board[row][col]) + '|'
        print(row_text)
    print(horizontal_line)
```

```
In [161... modded_ex_3_board = create_board(3)
draw_board(modded_ex_3_board)
```

```
    1  2  3
+---+---+---+
A |   |   |   |
+---+---+---+
B |   |   |   |
+---+---+---+
C |   |   |   |
+---+---+---+
```

Exercise 7: Write a function that takes a board, player number, and location specified as in exercise 6 and then calls exercise 5 to correctly modify the board.

```
In [134... def place_by_label(game_board, player_number, location):
    letter = location[0].upper()
```

```

number = location[1:]

row = ord(letter) - ord('A')
col = int(number) - 1

return modify_game_board(game_board, player_number, (row, col))

```

In [135...

```

ex_7_board = create_board(3)

print(place_by_label(ex_7_board, 1, 'A2'))
print(place_by_label(ex_7_board, 2, 'C1'))
print(place_by_label(ex_7_board, 2, 'A2')) # Occupied cell
draw_board(ex_7_board)

```

True

True

False

```

    1   2   3
+---+---+---+
A |   | X |   |
+---+---+---+
B |   |   |   |
+---+---+---+
C | O |   |   |
+---+---+---+

```

Exercise 8: Write a function is called with a board and player number, takes input from the player using python's `input`, and modifies the board using your function from exercise 7. Note that you should keep asking for input until you have gotten a valid input that results in a valid move.

In [136...

```

def get_player_move(game_board, player_number):
    while True:
        location = input(f'Player {player_number}, enter your move (e.g. A2): ').strip().upper()

        # Validate format.
        if len(location) < 2 or not location[0].isalpha() or not location[1:].isdigit():
            print('Invalid format. Must follow this format: A2 for example.')
            continue

        if place_by_label(game_board, player_number, location):
            return location
        else:
            print('Invalid move. Try again.')

```

In [137...

```

ex_8_board = create_board(3)
draw_board(ex_8_board)
get_player_move(ex_8_board, 1)
draw_board(ex_8_board)

```

```

      1   2   3
+---+---+---+
A |   |   |   |
+---+---+---+
B |   |   |   |
+---+---+---+
C |   |   |   |
+---+---+---+
      1   2   3
+---+---+---+
A | X |   |   |
+---+---+---+
B |   |   |   |
+---+---+---+
C |   |   |   |
+---+---+---+

```

Exercise 9: Use all of the previous exercises to implement a full tic-tac-toe game, where an appropriate board is drawn, 2 players are repeatedly asked for a location coordinates of where they wish to place a mark, and the game status is checked until a player wins or a draw occurs.

```

In [140... def play_game(board_size=3):
    game_board = create_board(board_size)
    current_player = 1

    while True:
        draw_board(game_board)

        get_player_move(game_board, current_player)

        state = check_game_state(game_board)
        if state != -1:
            draw_board(game_board)
            if state == 0:
                print('The game is a draw.')
            else:
                print(f'Player {state} wins!')
            break
        current_player = 2 if current_player == 1 else 1

```

```

In [141... play_game(3)

```

```

      1   2   3
+---+---+---+
A |   |   |   |
+---+---+---+
B |   |   |   |
+---+---+---+
C |   |   |   |
+---+---+---+
      1   2   3
+---+---+---+
A | X |   |   |
+---+---+---+
B |   |   |   |
+---+---+---+
C |   |   |   |
+---+---+---+
      1   2   3
+---+---+---+
A | X |   |   |
+---+---+---+
B |   | 0 |   |
+---+---+---+
C |   |   |   |
+---+---+---+
      1   2   3
+---+---+---+
A | X |   |   |
+---+---+---+
B | X | 0 |   |
+---+---+---+
C |   |   |   |
+---+---+---+
      1   2   3
+---+---+---+
A | X |   |   |
+---+---+---+
B | X | 0 |   |
+---+---+---+
C |   |   |   |
+---+---+---+
      1   2   3
+---+---+---+
A | X |   |   |
+---+---+---+
B | X | 0 |   |
+---+---+---+
C | X |   | 0 |
+---+---+---+
Player 1 wins!
```

Exercise 10: Test that your game works for 5x5 Tic Tac Toe.

```
In [142... play_game(5)
```


	1	2	3	4	5
A					
B					
C					
D					
E					

	1	2	3	4	5
A	X				
B					
C					
D					
E					

	1	2	3	4	5
A	X				
B					
C		0			
D					
E					

	1	2	3	4	5
A	X	X			
B					
C		0			
D					
E					

	1	2	3	4	5
A	X	X			
B			0		
C		0			
D					
E					

A		X		X		X					
	+	-	+	-	+	-	+	-	+	-	+
B						0					
	+	-	+	-	+	-	+	-	+	-	+
C				0							
	+	-	+	-	+	-	+	-	+	-	+
D											
	+	-	+	-	+	-	+	-	+	-	+
E											
	+	-	+	-	+	-	+	-	+	-	+
		1		2		3		4		5	

	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
A		X		X		X									
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
B						0									
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
C		0		0											
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
D															
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
E															
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
		1		2		3		4		5					

	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
A		X		X		X									
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
B						0									
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
C		0		0				X							
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
D															
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
E															
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
		1		2		3		4		5					

	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
A		X		X		X									
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
B		0				0									
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
C		0		0				X							
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
D															
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
E															
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
		1		2		3		4		5					

	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
A		X		X		X				X					
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
B		0			0										
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
C		0		0				X							
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
D															
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
E															
	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
		1		2		3		4		5					

$$\begin{array}{cccccccc}
 & + & - & - & + & - & - & + \\
 A & | & X & | & X & | & & | \\
 & + & - & - & + & - & - & +
 \end{array}$$

```

B | 0 | 0 | 0 |   |   |
+---+---+---+---+
C | 0 | 0 |   | X |   |
+---+---+---+---+
D |   |   |   |   |   |
+---+---+---+---+
E |   |   |   |   |   |
+---+---+---+---+
  1   2   3   4   5
+---+---+---+---+
A | X | X | X | X | X |
+---+---+---+---+
B | 0 | 0 | 0 |   |   |
+---+---+---+---+
C | 0 | 0 |   | X |   |
+---+---+---+---+
D |   |   |   |   |   |
+---+---+---+---+
E |   |   |   |   |   |
+---+---+---+---+

```

Player 1 wins!

Exercise 11: (Advanced / Challenge) Develop a version of the game where one player is the computer. Note that you don't need to do an extensive search for the best move. You can have the computer simply protect against loosing and otherwise try to win with straight or diagonal patterns. (Extra 10 Points)

In []: *# Write your solution here*

In []: *# Test your solution here*